JavaScript `this`, Arrow Functions, call, apply, bind

1. The `this` Keyword in JavaScript

`this` refers to the object that is executing the current function. Its value depends on how the function is called.

Examples:

```
// Global scope (non-strict)
function test() {
  console.log(this); // window (in browsers)
}
test();

// Strict mode
"use strict";
function testStrict() {
  console.log(this); // undefined
}
testStrict();
```

2. `this` Inside Objects

```
const item = {
  title: "Ball",
  getItem() {
    console.log(this); // refers to 'item'
  },
};

item.getItem();
// { title: 'Ball', getItem: [Function: getItem] }
```

3. `this` in Classes

```
class Item {
  title = "Ball";
  getItem() {
    console.log("this1:", this); // instance of Item

    function inner() {
      console.log("this:", this); // undefined in strict mode (default in classes)
    }
    inner();
  }
}

const obj = new Item();
obj.getItem();
// this1: Item { title: 'Ball' }
// this: undefined
```

4. Arrow Functions and `this`

Arrow functions do not have their own `this`. They inherit `this` from the enclosing lexical scope.

```
class Item {
  title = "Ball";
  getItem() {
    const inner = () => {
      console.log("this:", this); // refers to class instance
    };
    inner();
  }
}

const obj = new Item();
obj.getItem();
// this: Item { title: 'Ball' }
```

5. `call`, `apply`, and `bind`

These methods let you explicitly set the value of `this` when invoking a function.

call - invokes immediately, arguments are comma-separated:

```
function greet(greeting, name) {
  console.log(greeting + " " + name + ", from " + this.place);
}

const obj = { place: "Bangalore" };
greet.call(obj, "Hello", "Alisha");
// Hello Alisha, from Bangalore
```

apply - invokes immediately, arguments passed as array:

```
greet.apply(obj, ["Hi", "Anjum"]);
// Hi Anjum, from Bangalore
```

bind - returns new function with bound `this`, call it later:

```
const greetFromBlr = greet.bind(obj, "Hey");
greetFromBlr("Alisha");
// Hey Alisha, from Bangalore
```

You can also partially apply arguments using bind:

```
function multiply(a, b) {
  return a * b;
}
const double = multiply.bind(null, 2);
console.log(double(5)); // 10
```

Comparison Table:

Method	Invokes immediately?	Arguments format	Returns
call	Yes	Comma-separated list	Function result
apply	Yes	Array of arguments	Function result
bind	No	Comma-separated list	New function with bound this